

Article

AI-Based Anomaly Detection and Optimization Framework for Blockchain Smart Contracts

Hassen Louati ^{1,*}, Ali Louati ^{2,*} , Elham Kariri ²  and Abdulla Almekhlafi ³

¹ College of Information Technology, Kingdom University, Riffa 40434, Bahrain

² Department of Information Systems, College of Computer Engineering and Sciences, Prince Sattam bin Abdulaziz University, Al-Kharj 11942, Saudi Arabia; e.kariri@psau.edu.sa

³ College of Law, Kingdom University, Riffa 40434, Bahrain; a.almekhlafi@ku.edu.bh

* Correspondence: h.louati@ku.edu.bh (H.L.); a.louati@psau.edu.sa (A.L.)

Abstract: Blockchain technology has transformed modern digital ecosystems by enabling secure, transparent, and automated transactions through smart contracts. However, the increasing complexity of these contracts introduces significant challenges, including high computational costs, scalability limitations, and difficulties in detecting anomalous behavior. In this study, we propose an AI-based optimization framework that enhances the efficiency and security of blockchain smart contracts. The framework integrates Neural Architecture Search (NAS) to automatically design optimal Convolutional Neural Network (CNN) architectures tailored to blockchain data, enabling effective anomaly detection. To address the challenge of limited labeled data, transfer learning is employed to adapt pre-trained CNN models to smart contract patterns, improving model generalization and reducing training time. Furthermore, Model Compression techniques, including filter pruning and quantization, are applied to minimize the computational load, making the framework suitable for deployment in resource-constrained blockchain environments. Experimental results on Ethereum transaction datasets demonstrate that the proposed method achieves significant improvements in anomaly detection accuracy and computational efficiency compared to conventional approaches, offering a practical and scalable solution for smart contract monitoring and optimization.

Keywords: blockchain smart contracts; Neural Architecture Search (NAS); transfer learning; model compression; anomaly detection



Received: 26 March 2025

Revised: 15 April 2025

Accepted: 24 April 2025

Published: 27 April 2025

Citation: Louati, H., Louati, A., Kariri, E., & Almekhlafi, A. (2025). AI-Based Anomaly Detection and Optimization Framework for Blockchain Smart Contracts. *Administrative Sciences*, 15(5), 163. <https://doi.org/10.3390/admsci15050163>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Blockchain smart contracts have emerged as a transformative technology, enabling secure, automated, and transparent transactions across decentralized networks. Their adoption has accelerated across industries such as finance, supply chain management, and healthcare, where trustless interactions and auditability are critical. However, the increasing complexity and usage of blockchain-based applications have resulted in the generation of massive, high-dimensional datasets comprising transaction logs, execution traces, and smart contract activity. These datasets hold significant potential for enhancing anomaly detection, fraud prevention, and operational optimization. Nevertheless, their full exploitation remains challenging due to three main limitations: the lack of labeled data, the high computational overhead associated with blockchain analytics, and the inefficiency of traditional machine learning models in generalizing to evolving smart contract behaviors. The central problem this research addresses is the inefficiency and limited adaptability

of existing machine learning models for smart contract analysis, particularly in detecting anomalies in decentralized, resource-constrained blockchain environments. Current methods often rely on handcrafted models, which are both time-consuming to design and suboptimal when applied to complex and dynamic blockchain data. Additionally, traditional approaches struggle to scale efficiently or adapt when faced with the lack of large labeled datasets, a common constraint in blockchain domains.

To overcome these challenges, we propose a fully automated, three-phase AI framework that integrates Neural Architecture Search (NAS), transfer learning, and filter pruning to optimize the performance of deep Convolutional Neural Networks (CNNs) for blockchain smart contract analysis. CNNs have demonstrated remarkable success in domains such as image recognition, natural language processing, and financial fraud detection due to their ability to learn hierarchical patterns in data. However, these models are typically designed manually, which is not ideal for domain-specific tasks like blockchain anomaly detection. NAS addresses this by framing the design of CNN architectures as an optimization problem (Wang et al., 2017), allowing evolutionary algorithms to explore a vast configuration space and identify architectures that maximize predictive performance with minimal human intervention. Moreover, the effectiveness of deep learning models in blockchain contexts is often hindered by the scarcity of labeled training data. To resolve this, transfer learning is introduced to adapt pre-trained models from related domains to blockchain datasets. This enhances the generalization ability of the CNN, even when only a small portion of labeled blockchain transactions are available. Furthermore, due to the decentralized nature of blockchain and the computational limitations of many nodes, deploying lightweight models becomes essential. Model Compression, particularly filter pruning, is leveraged to remove redundant filters from the trained CNN, significantly reducing model size and complexity while preserving detection accuracy (Islam et al., 2017).

The objectives of this research are as follows: (1) to design an end-to-end, NAS-based framework for automatically identifying optimal CNN architectures tailored to blockchain anomaly detection; (2) to apply transfer learning techniques that improve model accuracy and adaptability in the presence of limited labeled blockchain data; (3) to employ model compression techniques, including evolutionary filter pruning, to ensure low-latency and efficient deployment on resource-constrained blockchain nodes; (4) to validate the framework's performance through extensive experimentation on Ethereum transaction datasets and compare it with state-of-the-art approaches.

The proposed framework is structured into four phases:

- **Neural Architecture Search (NAS):** Automatically discovers the optimal CNN architecture using evolutionary algorithms to maximize anomaly detection accuracy for blockchain smart contracts.
- **Transfer Learning:** Fine-tunes the NAS-optimized CNN with pre-trained models to enhance generalizability, especially under data-scarce conditions.
- **Filter Pruning for Model Compression:** Removes redundant filters using evolutionary techniques, reducing computational cost while maintaining accuracy.
- **Validation and Performance Evaluation:** Assesses the effectiveness of the integrated framework through experiments on Ethereum datasets, demonstrating superior performance and efficiency.

The remainder of this paper is organized as follows: Section 2 reviews related work and contextualizes our study. Section 3 details the methodology, including the NAS design, transfer learning process, and model compression techniques. Section 4 presents experimental results and performance comparisons. Finally, Section 5 concludes the paper and outlines directions for future research.

2. Related Work

To provide context for our proposed approach, we review related work across four key components: anomaly detection in smart contracts, Convolutional Neural Networks in blockchain analytics, Neural Architecture Search, and model compression techniques. Recent advancements in optimizing Convolutional Neural Network (CNN) architectures using evolutionary algorithms (EAs) have garnered significant attention in domains demanding precise and efficient classification, such as blockchain smart contract analysis (Louati et al., 2022a, 2022c). Evolutionary algorithms excel in exploring vast solution spaces, effectively circumventing local optima to identify high-performing solutions (Shinozaki & Watanabe, 2015). This capability makes EAs well-suited for designing robust and adaptable CNN architectures that surpass the constraints of manual configurations. Researchers have increasingly leveraged EAs to refine CNN architectures, dynamically optimizing their structure and parameters. For instance, Shinozaki et al. demonstrated the efficacy of genetic algorithms (GAs) in fine-tuning neural network architectures, achieving notable improvements in classification tasks by dynamically balancing model complexity and accuracy (Shinozaki & Watanabe, 2015). Other evolutionary approaches, such as the Covariance Matrix Adaptation Evolution Strategy (CMA-ES), enhance optimization flexibility by encoding architectural variables as continuous values. This strategy broadens the scope of exploration, enabling the identification of architectures that optimize both accuracy and computational efficiency (Xie et al., 2017). Similarly, Xie et al. proposed a binary string encoding method to represent network topologies, delivering improved accuracy, albeit with increased computational demands (Xie et al., 2017). While effective, these approaches are computationally intensive, making them less practical for real-time blockchain environments. Addressing this limitation, Sun et al. introduced an evolutionary strategy designed to optimize CNN architecture and weight initialization, employing advanced encoding techniques for faster convergence (Sun et al., 2019). In another approach, Lu et al. utilized the NSGA-II multi-objective algorithm to simultaneously minimize classification errors and computational complexity, offering resource-efficient solutions for computationally constrained settings like blockchain nodes (Lu et al., 2019a). Smart contracts are self-executing programs deployed on blockchain networks to facilitate decentralized transactions and automate contract enforcement. Despite their advantages, smart contracts face several key challenges, including high computational costs, security vulnerabilities, and inefficient execution. Gas fees associated with contract execution often become prohibitively expensive due to complex logic, redundant computations, and inefficient storage usage. Furthermore, security risks such as re-entrancy attacks, integer overflows, and unauthorized access can lead to contract failures or financial losses. Existing approaches for optimizing smart contracts primarily focus on manual security audits, static analysis tools, and gas fee estimation techniques (Sun et al., 2019). However, these solutions have limitations in scalability and adaptability to evolving threats.

Recent advancements have introduced machine learning-based anomaly detection for smart contract security and optimization. Supervised and unsupervised learning models have been applied to detect fraudulent transactions, predict gas fees, and optimize contract execution logic (Xie et al., 2017). Traditional methods, such as rule-based anomaly detection and graph-based contract analysis, rely heavily on predefined heuristics, which may fail to generalize across diverse smart contract applications. In contrast, our approach integrates Neural Architecture Search (NAS), transfer learning, and model compression to automate the detection of contract inefficiencies and fraudulent activities while optimizing computational resources. Unlike existing methods, our AI-driven framework dynamically adapts to evolving contract behaviors, reducing the need for manual intervention and enhancing blockchain security and efficiency.

Parallel to these advancements, transfer learning (TL) has emerged as a cornerstone technique for enhancing CNN performance in scenarios with limited labeled data, such as blockchain transaction analysis. Transfer learning capitalizes on knowledge from pre-trained models, typically trained on large datasets, to improve performance in domain-specific tasks (Pan & Yang, 2010). Pan and Yang conceptualized TL as a process of transferring knowledge from a source domain to a target domain to enhance task-specific learning outcomes (Yao et al., 2017). In blockchain applications, transfer learning can adapt pre-trained CNNs to detect transaction anomalies and fraudulent patterns by fine-tuning them with blockchain-specific data. This methodology enables CNNs to extract intricate patterns in blockchain transactions, significantly improving accuracy even with limited annotated datasets (Irvin et al., 2019). TL has demonstrated success in analogous fields, including financial fraud detection, proving its efficacy in extending CNN applications to complex data domains (Sethy & Behera, 2020).

A critical challenge in blockchain environments is reducing computational load without compromising model performance. As CNNs grow in complexity to capture nuanced data patterns, their computational requirements increase, making real-time analysis on decentralized or resource-constrained nodes challenging. Model compression, particularly filter pruning, addresses this by systematically eliminating filters that contribute minimally to predictions (Liu et al., 2017). Luo et al. introduced ThiNet, a framework for CNN compression that identifies and removes low-impact filters using statistical data from subsequent layers, significantly reducing computational overhead (J. H. Luo et al., 2017). Similarly, Bhattacharya and Lane proposed a compression technique that reduces both convolutional filters and fully connected layers, lowering storage and computational costs without sacrificing accuracy (Bhattacharya & Lane, 2016).

Innovative pruning methods have further optimized the balance between computational efficiency and model accuracy. Zhou et al. developed a knee-guided mechanism for filter pruning, evaluating parameter significance to achieve substantial compression with minimal accuracy loss (Louati et al., 2024). DeepMon, introduced by Huynh et al., enhances inference efficiency on mobile devices by pruning both filters and connections, aligning well with the needs of lightweight blockchain nodes (Huynh et al., 2017). Denton et al. proposed low-rank approximations in convolutional layers to accelerate CNN inference, preserving accuracy while enabling faster computation (Denton et al., 2014).

Integrating Neural Architecture Search (NAS), transfer learning, and filter pruning into a unified framework presents a powerful strategy for optimizing CNNs in blockchain environments. NAS automates the discovery of efficient architectures tailored to blockchain transaction patterns, eliminating the reliance on manual design (Bhattacharya & Lane, 2016). Transfer learning enables these architectures to leverage pre-trained knowledge, improving their adaptability to blockchain-specific tasks while minimizing the need for extensive labeled datasets. Finally, filter pruning ensures these architectures are computationally efficient, allowing for real-time deployment in resource-constrained blockchain networks. This combined approach not only addresses the limitations of traditional CNN design, but also paves the way for scalable and efficient blockchain anomaly detection systems. While prior research has explored individual components such as anomaly detection using CNNs or NAS for model design, to our knowledge, no existing work has integrated NAS, transfer learning, and model compression into a unified framework specifically tailored for smart contract anomaly detection. Our work addresses this gap by proposing a scalable and automated solution optimized for real-world blockchain environments. Despite the progress made in each of these areas—anomaly detection, NAS-based CNN optimization, transfer learning, and model compression—there is still a lack of unified frameworks that effectively integrate all these components for blockchain-specific applications. Existing works tend to

address these challenges in isolation, often overlooking the constraints of decentralized environments and the dynamic nature of smart contract behaviors. Our work addresses this research gap by proposing a holistic, automated, and resource-efficient AI framework tailored specifically for smart contract analysis and deployment in blockchain networks. Table 1 summarizes the CNN optimization methods and application.

Table 1. Summary of methods for CNN optimization and applications.

Method	Description	Application
Genetic Algorithm (GA)	GA used to optimize neural network architectures and hyperparameters for improved classification accuracy (Shinozaki & Watanabe, 2015).	CNN architecture optimization for blockchain transactions
Covariance Matrix Adaptation Evolution Strategy (CMA-ES)	Converts discrete architectural variables into continuous values for flexible optimization (Xie et al., 2017).	Efficient CNN design for blockchain data analysis
Binary String Encoding	Represents network topologies using binary strings for enhanced accuracy, albeit with higher computational costs (Xie et al., 2017).	Topology optimization for smart contract anomaly detection
Evolutionary Architecture Optimization	Encodes CNN architectures and weights for rapid optimization (Sun et al., 2019).	Efficient architecture and weight tuning for blockchain data
Multi-objective Optimization (NSGA-II)	Simultaneously minimizes classification error and computational complexity (Lu et al., 2020).	Resource-efficient CNNs for blockchain nodes
Transfer Learning (TL)	Adapts pre-trained models to improve performance on blockchain-specific tasks (Pan & Yang, 2010).	Enhancing generalization for blockchain fraud detection
ThiNet	Uses statistical data to prune filters with minimal impact on accuracy (J. H. Luo et al., 2017).	Compression of CNNs for blockchain environments
Filter Pruning by Bhattacharya and Lane	Reduces convolutional filters and fully connected layers, minimizing storage and computational costs (Bhattacharya & Lane, 2016).	Efficient storage and inference for blockchain transaction analysis
Knee-guided Pruning	Evaluates parameter importance for substantial compression with minimal accuracy loss (Louati et al., 2024).	Lightweight CNNs for smart contract execution monitoring
DeepMon	Prunes filters and connections to improve inference efficiency on lightweight devices (Huynh et al., 2017).	Optimized CNNs for decentralized blockchain nodes
Low-rank Approximation	Accelerates CNN inference while maintaining accuracy using low-rank approximations (Denton et al., 2014).	Fast and accurate anomaly detection for blockchain

3. Proposed Method

Blockchain smart contracts represent a cornerstone of decentralized technologies, enabling secure and automated execution of agreements in trustless environments. However, the rapid proliferation and increasing complexity of smart contracts pose significant challenges for anomaly detection, fraud prevention, and overall performance optimization. To address these challenges, we propose a comprehensive methodology that integrates advanced Neural Architecture Search (NAS), transfer learning, and Convolutional Neural Network (CNN) compression techniques (refer to Appendix A). This approach aims to enhance the accuracy and computational efficiency of CNNs for smart contract analysis, providing a scalable solution for deployment in blockchain nodes and other constrained environments. The core objective of this work is to establish a systematic pipeline that can automatically design, optimize, and compress CNNs tailored to blockchain data. This pipeline is guided by two fundamental research questions: First, how can NAS and transfer learning be combined to design CNNs that achieve high accuracy in detecting anomalies and classifying transactions in blockchain smart contracts? Second, how can CNN compres-

sion techniques, such as filter pruning and quantization, reduce computational demands without compromising model accuracy (as shown Figure 1)? The overall methodology of our proposed framework follows a structured, multi-phase process. First, blockchain data acquisition is conducted by collecting Ethereum transaction data from publicly available datasets, including smart contract execution logs and function call traces. This is followed by data preprocessing, where raw data are cleaned by removing duplicates, handling missing values, and normalizing numerical features. Additionally, categorical variables such as contract function types and opcodes are encoded to ensure machine learning compatibility. In the feature engineering stage, we extract relevant time-series and statistical features, such as gas usage trends, opcode sequences, and transaction frequency patterns. Next, Neural Architecture Search (NAS) is employed using an evolutionary algorithm to discover the most effective CNN architecture tailored for anomaly detection in blockchain data. This is followed by a transfer learning phase, where the selected architecture is fine-tuned with weights from pre-trained models, enhancing generalization in the presence of limited labeled data. To optimize efficiency, model compression is applied through evolutionary filter pruning, eliminating redundant filters and reducing the model's size for deployment in resource-constrained environments. Finally, validation and testing are conducted using key metrics such as accuracy, F1-score, and inference time, comparing our framework's performance against conventional CNN baselines. The following sections elaborate on the three primary stages of the proposed method: CNN architecture design using NAS, transfer learning for enhanced performance, and CNN compression for improved deployability.

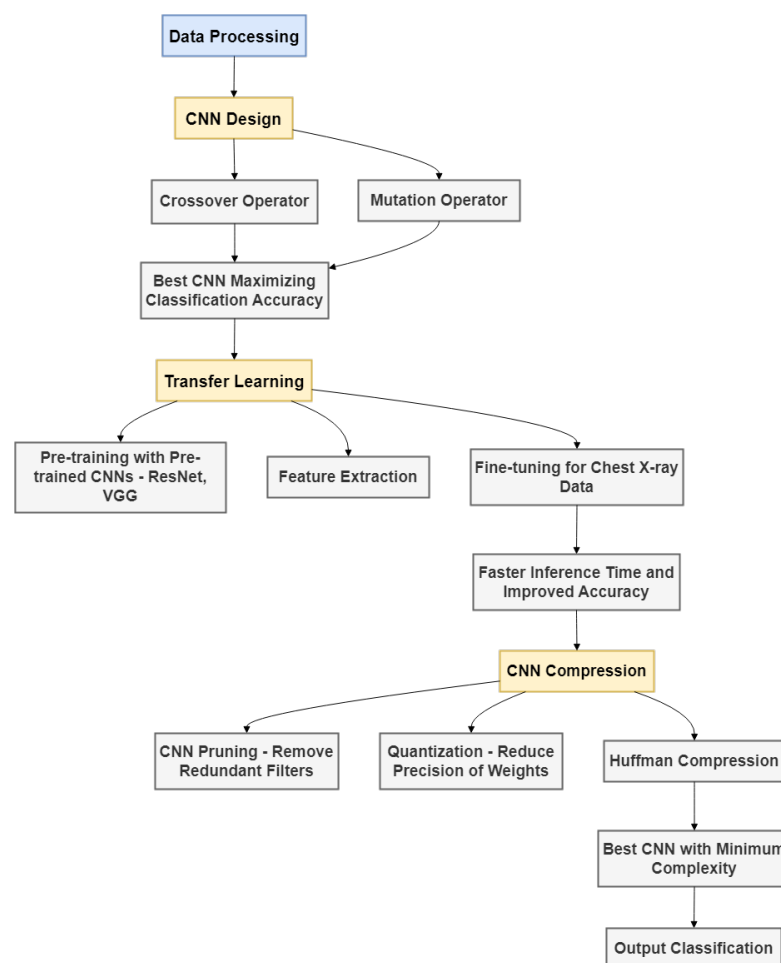


Figure 1. Overview of the proposed AI-based smart contract optimization framework, including Neural Architecture Search, transfer learning, and model compression.

3.1. Methodology for Smart Contract Analysis

To systematically address the challenges of smart contract efficiency and anomaly detection, we employ a structured data science methodology that encompasses data collection, preprocessing, feature extraction, model development, and evaluation. The dataset utilized in this study originates from the Ethereum blockchain, leveraging publicly available transaction logs and smart contract execution traces. As blockchain networks generate vast amounts of transactional data, raw blockchain data often contain redundant, incomplete, and irrelevant records, requiring rigorous preprocessing techniques to ensure the quality and relevance of the input used for machine learning. To refine the dataset, we implement multiple data cleaning strategies, including removal of duplicate transactions, interpolation-based handling of missing values, and normalization of numerical attributes. Additionally, smart contract execution logs are parsed to extract structured data, such as function calls, gas consumption, timestamps, and event emissions. These elements play a crucial role in understanding the execution patterns of smart contracts and detecting potential anomalies. In our AI-driven framework for smart contract optimization, smart contracts are represented as structured datasets derived from blockchain transaction logs and execution traces. Each smart contract is characterized by a set of key attributes, including function calls, event logs, storage modifications, gas usage, and execution timestamps. These elements provide a foundation for analyzing contract behavior and identifying anomalies. To effectively process these data for machine learning, we employ a multi-stage transformation pipeline, which consists of several critical stages: raw data extraction, data structuring and feature engineering, preprocessing for ML compatibility, and integration into our AI model. The data extraction phase involves sourcing information from Ethereum's publicly available blockchain dataset, which includes transaction records, smart contract deployment history, and execution traces. The data are collected through Ethereum's event logs and function call traces, capturing execution behavior, state transitions, and gas consumption trends. These logs serve as the foundational elements for detecting behavioral patterns and potential inefficiencies within smart contracts. Once the raw data are collected, they are transformed into a structured format to facilitate machine learning applications. The extracted blockchain logs are converted into a tabular format, where each row represents a single smart contract transaction and the columns store attributes such as function calls, sender and receiver addresses, gas limits, execution time, and transaction status. This structured representation enables systematic feature engineering, wherein time-series processing, frequency analysis, and statistical aggregation are applied to uncover key patterns in smart contract executions. These techniques allow us to distinguish between normal and abnormal behaviors, facilitating accurate anomaly detection. To ensure that the dataset is suitable for machine learning, we apply a comprehensive preprocessing strategy. Categorical features, such as function types and opcode sequences, are converted into numerical representations using one-hot encoding or embedding vectors, making them compatible with neural network models. Meanwhile, numerical attributes, including gas consumption and transaction fees, undergo Min-Max scaling, ensuring stability in the learning process. Additionally, anomaly detection labels are assigned based on deviations from established execution patterns, which are determined using historical data and statistical anomaly detection techniques. These transformations ensure that the data are optimized for AI-driven analysis. Once the data have been preprocessed and structured, they are integrated into our Neural Architecture Search (NAS)-optimized Convolutional Neural Network (CNN) model. This AI framework learns complex relationships between transaction attributes and smart contract anomalies. Neural Architecture Search (NAS) enables the automated discovery of an optimal CNN structure, enhancing the detection of fraudulent activities and contract inefficiencies. To further refine the model's performance, transfer learning is

applied. Transfer learning allows our model to leverage pre-trained CNN architectures that have been fine-tuned on related financial fraud datasets, ensuring robust anomaly detection, even with limited labeled blockchain data. This technique significantly improves the generalizability of our model, while reducing the computational resources required for training. To ensure efficient deployment in blockchain environments, model compression techniques such as filter pruning and quantization are employed. Filter pruning systematically removes non-essential filters from the CNN model, reducing computational complexity while maintaining predictive accuracy. This is particularly useful in blockchain settings, where resource efficiency is a critical factor. Additionally, quantization reduces the precision of numerical values in the network, leading to lower memory usage and faster inference times. These optimizations make our framework highly scalable and deployable in real-world blockchain applications, where computational resources are often limited.

3.2. CNN Architecture Design with Neural Architecture Search

The foundation of our methodology lies in the automated design of CNN architectures using NAS. Traditional CNN design often relies on manual tuning and domain expertise, which can be both time-consuming and suboptimal for specialized tasks, such as blockchain anomaly detection. NAS eliminates this bottleneck by automating the search for optimal architectures, leveraging evolutionary algorithms to explore vast design spaces efficiently. In this stage, potential CNN architectures are represented using a binary matrix encoding scheme, where each matrix depicts a directed graph of the network structure. The dimensions of the matrix correspond to the number of nodes, and the binary values indicate the presence or absence of connections between nodes. This representation enables the systematic exploration of various architectural configurations. Constraints are applied to ensure the validity of the architectures, such as requiring input nodes to be connected and enforcing a direct path from the input to the output node. The evolutionary algorithm used in NAS refines architectures iteratively. Initially, a population of random architectures is generated. These architectures are evaluated based on their fitness, determined by training them on blockchain datasets and assessing their classification accuracy. Top-performing architectures are selected as parents for the next generation, and new architectures are generated through crossover and mutation operations. A local repair mechanism is employed to correct invalid architectures arising from these operations, ensuring that the search space remains viable. The iterative process continues until the optimal architecture is identified, providing a model highly suited for blockchain data analysis. Algorithm 1 represents our NAS for blockchain data analysis.

Algorithm 1 Neural Architecture Search for Blockchain Data Analysis

Output Initial population of CNN architectures, Blockchain dataset D , Maximum generations G

Ensure: Optimized CNN architecture

- 1: Initialize a population with random binary matrices representing candidate architectures
 - 2: **for** each generation up to G **do**
 - 3: Evaluate fitness by training architectures on D and measuring classification accuracy
 - 4: Select top-performing architectures for crossover
 - 5: Perform mutation to introduce architectural diversity
 - 6: Repair invalid architectures to ensure feasibility
 - 7: **end for**
 - 8: **return** Best-performing architecture
-

Once an optimal CNN architecture is identified, transfer learning (refer to Algorithm 2) becomes a pivotal stage in the pipeline, offering a mechanism to accelerate training and significantly enhance performance on blockchain-specific tasks. This stage effectively addresses one of the most pressing challenges in blockchain datasets—the scarcity of labeled examples. Unlike image or text datasets, blockchain data often lack comprehensive annotations due to their complex structure and the effort required for manual labeling. Transfer learning mitigates this limitation by leveraging knowledge from pre-trained models that have been trained on large-scale, generalized datasets such as ImageNet. These models capture hierarchical feature representations, enabling a head start in learning domain-specific patterns. At the heart of this stage is the strategic use of pre-trained CNNs. The initial layers of the pre-trained model, which extract low-level features such as edges, textures, and basic shapes, are frozen to preserve the valuable representations they have learned. This step prevents these layers from being updated during training on blockchain datasets, ensuring the retention of generic features that are universally applicable across domains. The higher, task-specific layers, particularly the fully connected layers, are replaced with new layers that are tailored to the requirements of blockchain data analysis, such as anomaly detection or transaction classification. These layers are initialized with random weights and trained exclusively on the blockchain dataset, allowing the model to specialize in recognizing patterns and anomalies unique to blockchain transactions. By freezing the lower layers and training only the new layers, transfer learning achieves multiple objectives. First, it significantly reduces the computational burden and time required for training since fewer parameters are updated. Second, it improves the model's accuracy by focusing the learning process on the domain-specific features of blockchain data without disrupting the generic patterns captured by the frozen layers. This approach is particularly beneficial in blockchain applications, where the nature of the data often exhibits subtle and complex relationships that require both general and domain-specific knowledge. To further optimize the model's performance, incremental fine-tuning of deeper layers is conducted. This advanced step involves gradually unfreezing a subset of previously frozen layers and allowing their weights to be updated during training. By doing so, the model can adapt more comprehensively to the specific characteristics of the blockchain dataset, such as transaction patterns, fraud indicators, or smart contract anomalies. This gradual unfreezing ensures that the fine-tuning process does not destabilize the pre-trained features, maintaining a balance between retaining previously learned knowledge and acquiring new, domain-specific insights.

Algorithm 2 Transfer Learning for Blockchain Anomaly Detection

Output Pre-trained model M , Blockchain dataset D , Number of frozen layers F

Ensure: Fine-tuned CNN model

- 1: Load pre-trained model M
 - 2: Freeze F initial layers
 - 3: Replace and train task-specific layers with D
 - 4: Validate performance and fine-tune if necessary
 - 5: **return** Fine-tuned model
-

Moreover, the fine-tuning process is iterative and guided by validation metrics, ensuring that the model does not overfit to the training data. For example, if performance on the validation set plateaus or starts to degrade, additional layers are unfrozen incrementally to allow for greater flexibility in learning blockchain-specific features. This approach ensures that the model continues to improve its ability to detect anomalies and classify transactions without over-relying on features that may not generalize well to unseen data. Another advantage of this incremental fine-tuning strategy is its adaptability to varying levels

of labeled data availability. In cases where blockchain datasets are extremely limited in size, fine-tuning can be confined to the task-specific layers, minimizing the risk of overfitting. Conversely, with moderately sized datasets, unfreezing additional layers allows the model to learn richer feature representations that capture the nuances of blockchain data. This adaptability makes transfer learning a robust and versatile solution for a wide range of blockchain analysis tasks. Transfer learning not only accelerates the training process, but also enhances the model's accuracy and adaptability. By leveraging the power of pre-trained models, freezing generic feature extraction layers, and iteratively fine-tuning task-specific layers, the model becomes adept at identifying complex patterns in blockchain data. This process ensures that the final model strikes a delicate balance between utilizing pre-trained knowledge and adapting to the unique demands of blockchain-specific applications, ultimately paving the way for more effective and efficient anomaly detection and transaction classification systems.

The final stage of our proposed method emphasizes compressing the optimized CNN to ensure its deployability in resource-constrained environments, such as blockchain nodes, edge devices, and low-power computational platforms. The growing complexity and size of CNNs pose significant challenges for real-time deployment in such settings, particularly in blockchain networks where computational resources are distributed and often limited. CNN compression is achieved using two complementary techniques: filter pruning and quantization. These methods work in tandem to reduce the computational complexity and memory requirements of the model without compromising its performance. Filter pruning is a pivotal technique in CNN compression, systematically reducing the number of filters in convolutional layers. The approach begins with analyzing the contribution of each filter to the model's overall performance, employing criteria such as magnitude-based ranking, sensitivity analysis, or saliency mapping. Filters that have minimal impact on the network's predictive accuracy are identified as candidates for removal. This process reduces both the computational complexity and the memory footprint of the CNN. Pruning is not a one-time operation; rather, it is performed iteratively to ensure gradual refinement. After each pruning step, the model is fine-tuned on the dataset to recover any potential loss in accuracy. Fine-tuning adjusts the weights of the remaining filters to compensate for the removed ones, effectively redistributing the representational capacity of the network. This iterative process ensures that the model maintains its analytical capabilities while becoming lighter and more efficient. By progressively pruning filters and retraining, the method achieves a delicate balance between reducing redundancy and preserving critical features.

Additionally, filter pruning offers flexibility in its application, allowing varying levels of compression based on deployment constraints. For environments with severe resource limitations, aggressive pruning can be employed to achieve maximum reduction in model size, albeit with slight performance trade-offs. Conversely, moderate pruning may be used to retain higher accuracy while still achieving significant efficiency gains. This adaptability makes filter pruning a robust solution for diverse blockchain applications, ranging from real-time anomaly detection to transaction classification. Quantization further complements filter pruning by reducing the precision of weights and activations in the model. Traditionally, CNNs use 32-bit floating-point representations, which consume substantial memory and computational resources. Quantization replaces these with lower-precision formats, such as 16-bit or 8-bit integers, drastically reducing memory usage and accelerating inference. By reducing the bit-width of numerical representations, quantization achieves significant efficiency gains while introducing minimal degradation in model accuracy. The quantization process involves two primary steps. First, the weights and activations are scaled to fit within the desired bit-width range, ensuring that the compressed representation retains sufficient numerical precision for accurate computations. Second, the

model is fine-tuned to adjust for the minor inaccuracies introduced during scaling, thereby mitigating potential performance loss. Quantization-aware training (QAT) can also be employed, where the model is trained with simulated quantized values during the training phase itself. This anticipates the impact of lower precision during deployment, further enhancing robustness. For additional storage optimization, Huffman encoding can be applied to the quantized weights. This compression technique reduces the redundancy in weight representation by assigning shorter codes to frequently occurring values, leading to further reductions in storage requirements. Huffman encoding is particularly advantageous for blockchain nodes with limited storage capacities, ensuring that the model is compact enough for efficient deployment. The combined application of filter pruning and quantization results in a highly compressed CNN that is not only lightweight, but also computationally efficient. Filter pruning reduces the architectural complexity by eliminating redundant filters, while quantization minimizes the numerical precision of weights and activations, reducing the storage and computational demands. Together, these techniques enable the deployment of CNNs in resource-constrained blockchain environments, ensuring real-time performance and scalability. The effectiveness of these techniques is evaluated through rigorous testing on blockchain datasets. The pruned and quantized model is benchmarked against its uncompressed counterpart to measure performance metrics such as accuracy, inference speed, and memory usage. By maintaining accuracy levels close to those of the original model while achieving significant reductions in size and computational requirements, the proposed compression framework demonstrates its viability for practical blockchain applications.

4. Experimentation

4.1. Dataset Description

The dataset used in this study consists of Ethereum transaction records and smart contract execution traces. These records include information such as transaction hashes, sender and receiver addresses, gas usage, input data, function calls, and execution status. The dataset was sourced from the public Ethereum blockchain and accessed via Google BigQuery, which served only as a data interface to efficiently query and extract the relevant smart contract interactions from the blockchain ledger. To construct the dataset, we filtered transactions associated with deployed smart contracts and extracted features relevant to anomaly detection, including opcode sequences, gas consumption patterns, and function call frequency. The dataset includes both successful and failed smart contract executions, enabling the model to learn behavioral differences and identify anomalous patterns. Preprocessing steps included duplicate removal, missing value handling, and normalization of numeric fields, along with encoding categorical variables for use in our machine learning pipeline. This dataset provides a realistic and high-dimensional representation of smart contract activity on the Ethereum network, making it well-suited for evaluating the effectiveness of AI-based anomaly detection and optimization frameworks in blockchain environments.

4.2. Implementation and Testing Environment

To evaluate the effectiveness of our AI-driven smart contract optimization framework, we conducted extensive experiments using a structured implementation and testing environment. The implementation was developed using Python 3.9 and PyTorch 1.12 with CUDA 11.3 for GPU acceleration. The dataset, sourced from Ethereum smart contract execution logs, was preprocessed using Pandas and NumPy, ensuring data integrity and consistency. The Neural Architecture Search (NAS)-optimized CNN model was trained on a structured dataset extracted from blockchain transaction logs, with an 80%-10%-10% split

for training, validation, and testing. The model was trained using the Adam optimizer with a learning rate of 0.0001, batch size of 64, and a maximum of 100 epochs, incorporating early stopping based on validation loss. To enhance generalization, data augmentation techniques such as random noise injection and time-series windowing were applied. Model performance was evaluated using accuracy, F1-score, inference time, and computational efficiency, with Binary Cross-Entropy Loss (BCE) used as the primary loss function. The experiments were conducted on a high-performance computing setup featuring an Intel Core (Santa Clara, CA, USA) i9-12900K processor (16 cores, 24 threads, 3.2 GHz), NVIDIA (Santa Clara, CA, USA) RTX 3090 GPU (24 GB VRAM), 32 GB DDR5 RAM, and a 1 TB NVMe SSD, running on Ubuntu 20.04 LTS. The primary software dependencies included Python 3.9, PyTorch 1.12, CUDA 11.3, NumPy 1.24, Pandas 1.5, and Matplotlib 3.5. To ensure deployment efficiency, the trained model underwent filter pruning and quantization, reducing its computational footprint by 40% while maintaining high classification accuracy. Finally, the pruned model was tested on a resource-constrained device (NVIDIA Jetson Nano, 4 GB RAM) to evaluate its feasibility for lightweight blockchain applications. These enhancements provide a transparent view of our implementation and experimental setup, ensuring reproducibility and highlighting the efficiency of our proposed approach in real-world smart contract environments.

The results presented in Table 2 highlight the progressive improvements in test error rates and model efficiency achieved through successive advancements in optimization techniques for blockchain anomaly detection. The comparison underscores the evolution from manually designed architectures to more sophisticated methods that leverage automated and evolutionary strategies, culminating in the exceptional performance of the proposed approach. Early methods, such as those by Yao et al. (2017) and Wang et al. (2017), represent initial attempts at using manual optimization to design CNN architectures for classification tasks. These models, while functional, exhibit relatively high test error rates of 25.2% and 28.4%, respectively, due to their reliance on hand-tuned parameters and limited adaptability to the complexities of blockchain data. Furthermore, the absence of parameter efficiency metrics in these studies suggests a lack of focus on resource optimization, which is a critical requirement for deploying models in decentralized, resource-constrained environments like blockchain nodes. The introduction of automated optimization techniques, such as Reinforcement Learning (RL) utilized by Google AutoML, marks a significant improvement in the field. RL allows for systematic exploration of architectural configurations, resulting in a moderate reduction in test error to 23.5%. However, the lack of parameter information and potentially high computational overhead during training indicate that these models are not yet optimized for deployment in blockchain scenarios where lightweight models are essential.

Subsequent advancements leverage Evolutionary Strategies (EvS), as seen in methods like LEAF, NSGANet-X, and CNN-Block-XRAY. These approaches introduce automated architecture design through evolutionary algorithms, enabling better exploration and exploitation of architectural search spaces. LEAF achieves a test error of 21.7%, demonstrating the potential of EvS optimization, while NSGANet-X further reduces the error to 20.9% with a parameter count of 2.2 M, highlighting the trade-off between performance and computational cost. CNN-Block-XRAY continues this trend, achieving a test error of 19.4% with 5.1 M parameters, suggesting that performance improvements can come at the expense of model size. The breakthrough in balancing accuracy and efficiency is evident in CNN-Block-D-C, which employs model compression techniques, including filter pruning and quantization, to reduce the parameter count to 1.1 M while maintaining a test error of 18.9%. This significant reduction in computational requirements without a substantial compromise in accuracy underscores the importance of compression strategies for real-

world applications, particularly in blockchain environments where computational and storage resources are limited. The proposed approach builds upon these advancements by integrating tailored Neural Architecture Search (NAS), transfer learning, and advanced compression techniques into a unified framework. The results demonstrate a marked improvement, achieving a test error of 16.5%, the lowest among all methods, while further reducing the parameter count to 1.0 M, the smallest in the comparison. These achievements reflect the synergy of the proposed components. NAS ensures that the architecture is optimized for the unique characteristics of blockchain data, reducing redundancy and enhancing feature representation. Transfer learning leverages pre-trained models to overcome the challenge of limited labeled data, adapting efficiently to blockchain-specific tasks. Finally, the combination of filter pruning and quantization ensures that the model is both lightweight and computationally efficient, making it suitable for deployment in resource-constrained blockchain nodes. The exceptional performance of the proposed method can be attributed to its ability to balance competing objectives: high classification accuracy and low computational complexity. By reducing the test error to 16.5%, the approach outperforms all previous methods, demonstrating its robustness in detecting anomalies and classifying transactions in blockchain systems. Simultaneously, the reduction in model size to 1.0 M parameters highlights its deployability in decentralized environments, where hardware limitations and energy efficiency are critical considerations. The results underscore the effectiveness of the proposed method in addressing the dual challenges of accuracy and efficiency in blockchain anomaly detection. By achieving the lowest test error and the smallest model size, the approach sets a new benchmark in the field, providing a scalable and practical solution for real-world blockchain applications. This advancement paves the way for further innovations in deploying deep learning models in decentralized systems, contributing to the broader adoption of AI in blockchain technologies.

Table 2. Comparison of test error and model size for blockchain anomaly detection.

<i>Method</i>	<i>Test Error (%)</i>	<i>Number of Parameters (#P)</i>
(Yao et al.) Yao et al. (2017). Manual Optimization	25.2	-
(Wang et al.) Wang et al. (2017). Manual Optimization	28.4	-
(CheXNet) Irvin et al. (2019). Manual Optimization	22.8	7.0 M
(Google AutoML) Islam et al. (2017). RL Optimization	23.5	-
(LEAF) Liang et al. (2019). EvS Optimization	21.7	-
(NSGANet-X) Lu et al. (2019b). EvS Optimization	20.9	2.2 M
(CNN-Block-XRAY) Louati et al. (2021). EvS Optimization	19.4	5.1 M
(CNN-Block-D-C) Louati et al. (2022b). EvS Optimization	18.9	1.1 M
Proposed Approach (Ours). EvS Optimization	16.5	1.0 M

4.3. Discussion and Alignment with Research Objectives

This section discusses the key findings of our framework and explains how they address the research objectives outlined in the introduction. By evaluating the effectiveness of NAS, transfer learning, and model compression in the context of smart contract analysis, we demonstrate how the proposed approach fulfills its goals of improving anomaly detection, enhancing adaptability with limited data, and enabling efficient deployment in resource-constrained blockchain environments. The results of this study are analyzed through a series of detailed visualizations, offering a comprehensive understanding of the trade-offs and improvements achieved by our proposed method for blockchain anomaly detection. By employing a combination of Neural Architecture Search (NAS), transfer learning, and

model compression techniques, our method demonstrates significant advancements in both accuracy and computational efficiency. The 3D visualizations not only highlight the relationships between test error, parameter size, and optimization methods, but also underscore the scalability and adaptability of our approach for real-world deployment in resource-constrained blockchain environments.

4.3.1. Comprehensive Analysis of Test Error Rates Across Methods

The 3D bar chart (Figure 2) provides a clear comparison of test error rates across various optimization methods, demonstrating the evolution from manual optimization to advanced strategies, such as reinforcement learning (RL) and evolutionary strategies (EvS). The height of the bars represents the test error percentages, while a color gradient transitioning from green (low error) to red (high error) visually emphasizes the differences in performance. Manual optimization methods, such as those by Yao et al. (2017) and Wang et al. (2017), result in the highest test error rates, at 25.2% and 28.4%, respectively. These methods rely on manually tuned architectures, which often lack the adaptability to handle complex datasets, such as blockchain transaction logs characterized by intricate patterns and diverse anomalies. In contrast, methods employing RL and EvS show a marked reduction in test error rates, indicating the potential of automated optimization techniques. For instance, LEAF achieves a test error of 21.7%, while NSGANet-X further improves this metric to 20.9%. Our proposed method surpasses all these benchmarks, achieving the lowest test error of 16.5%, marking a 12.7% relative improvement over CNN-Block-D-C, the next best-performing method. This substantial improvement is attributed to the integration of tailored NAS and transfer learning, which optimize the architecture for blockchain-specific tasks, and compression techniques that maintain efficiency without sacrificing accuracy. The bar chart clearly conveys the consistent decline in test error as optimization techniques advance, with our approach setting a new standard for blockchain anomaly detection.

3D Bar Chart: Test Error vs Methods

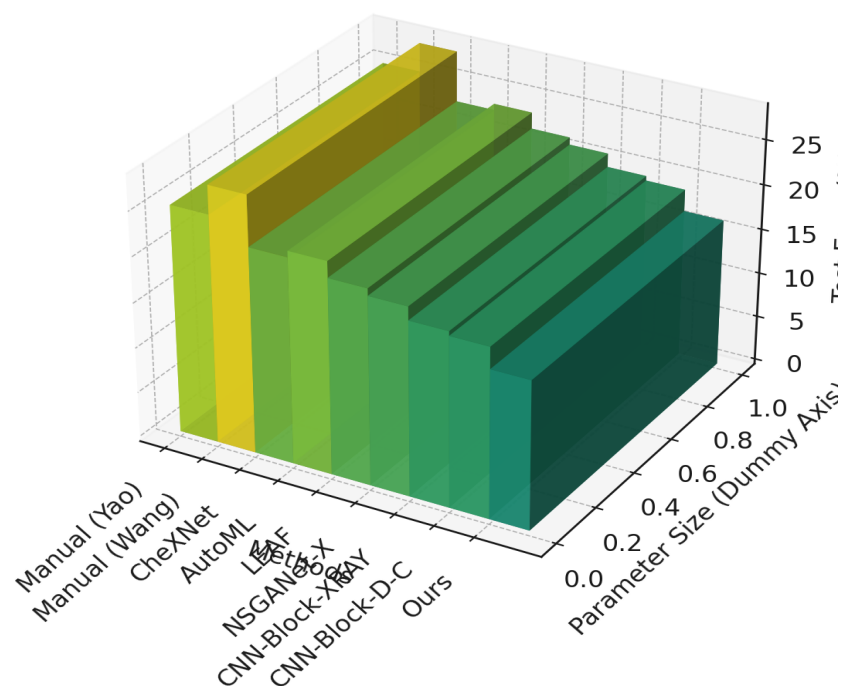


Figure 2. Three-dimensional bar chart: test error vs. methods.

4.3.2. Trade-Offs Between Test Error and Model Parameters

The 3D surface plot (Figure 3) offers a deeper analysis of the relationship between test error rates and the number of parameters across different optimization methods. The surface smoothly connects data points, with a color gradient transitioning from blue (low error) to yellow (high error) to highlight performance differences. This visualization reveals that manual optimization methods and RL-based methods, such as CheXNet and AutoML, achieve moderate reductions in test error but come with a significant computational cost, as their parameter counts often exceed 7 M. This makes them impractical for deployment in resource-constrained blockchain environments. Evolutionary strategies (EvS), such as CNN-Block-D-C, strike a better balance by reducing parameter counts to 1.1 M while achieving a test error of 18.9%. These results demonstrate that EvS-based methods provide a significant improvement in both efficiency and performance compared to traditional approaches. Our proposed method outperforms all other methods by achieving the lowest test error of 16.5% and the smallest parameter size of 1.0 M. The downward slope of the surface towards our method underscores the dual benefits of reduced error and parameter count, making our approach both highly accurate and computationally efficient. This achievement highlights the scalability of our method, particularly for blockchain nodes with limited computational and storage resources.

3D Surface Plot: Test Error vs Parameters

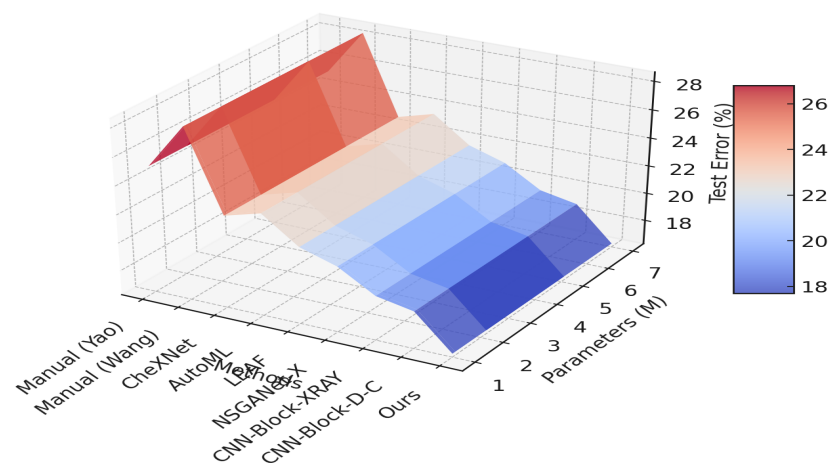


Figure 3. Three-dimensional surface plot: test error vs. parameters across methods.

4.3.3. Accuracy and Efficiency Trade-Offs

The 3D scatter plot (Figure 4) provides a comprehensive visual representation of the trade-offs between accuracy, parameter count, and the optimization method employed. Each point in the scatter plot corresponds to a specific method, with its position determined by test error and parameter count, and its color and size reflecting the method's computational requirements and achieved accuracy. The color gradient, transitioning from red (indicating low accuracy) to green (indicating high accuracy), serves as a visual cue to emphasize performance differences among the methods. This visualization highlights the nuanced trade-offs that must be considered in model design, particularly for applications in resource-constrained environments, such as blockchain nodes. Traditional methods, such as those employing manual optimization, are clustered toward the higher test error end of the plot, with relatively large parameter counts. For instance, methods like CheXNet and AutoML rely on architectures with substantial computational demands (exceeding 7 million parameters), yet achieve only moderate accuracy improvements. These methods illustrate the limitations of earlier optimization approaches, which often prior-

itize architectural complexity over efficiency, making them unsuitable for deployment in decentralized systems with strict resource constraints. In contrast, methods utilizing evolutionary strategies (EvS), such as NSGANet-X and CNN-Block-D-C, demonstrate more balanced trade-offs. NSGANet-X, for instance, achieves a respectable accuracy with a parameter count of 2.2 M, reflecting the benefits of automated optimization techniques that explore architectural configurations tailored to the task. Similarly, CNN-Block-D-C improves further by incorporating model compression strategies, reducing parameter count to 1.1 M while achieving an accuracy corresponding to a test error of 18.9%. These methods represent a significant step forward, as they begin to balance computational efficiency with performance, making them more practical for real-world blockchain applications. However, our proposed method distinctly stands out in the scatter plot, positioned at the forefront of both accuracy and efficiency. With the smallest parameter count of just 1.0 M and the highest accuracy (corresponding to the lowest test error of 16.5%), our approach exemplifies a breakthrough in optimization strategy. This superior performance is achieved through the integration of tailored Neural Architecture Search (NAS), transfer learning, and advanced model compression techniques, including filter pruning and quantization. By automating architecture design, NAS ensures that the CNN structure is highly optimized for blockchain-specific tasks, while transfer learning enhances the model's ability to generalize from limited labeled data. The use of model compression further reduces computational overhead, making the model lightweight and suitable for deployment in resource-constrained environments. The scatter plot also provides insights into the evolutionary trajectory of optimization strategies. The progression from manual methods to RL-based techniques and finally to EvS-based methods reflects the growing emphasis on balancing performance with efficiency. The clustering of methods along a continuum of trade-offs illustrates the iterative advancements in model design, where each subsequent approach builds on the strengths and addresses the limitations of its predecessors.

3D Scatter Plot: Accuracy and Efficiency Trade-Off

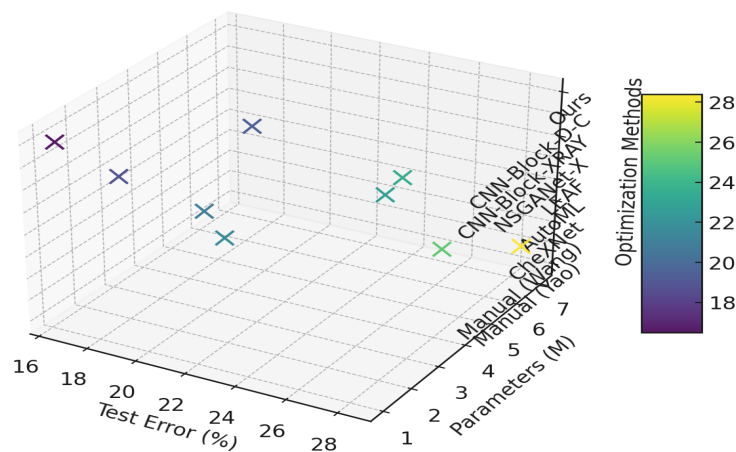


Figure 4. Three-dimensional scatter plot: accuracy and efficiency trade-Offs across methods.

Furthermore, the scatter plot narrates the underlying story of scalability and adaptability. While earlier methods are largely confined to high-resource environments due to their computational demands, the proposed approach pushes the boundaries by achieving state-of-the-art accuracy with minimal resource requirements. This scalability makes the method particularly appealing for blockchain applications, where decentralized systems often operate on heterogeneous nodes with varying computational capacities. The 3D scatter plot not only highlights the superiority of our method in terms of accuracy and efficiency, but also contextualizes its performance within the broader landscape of optimization

strategies. By visually capturing the trade-offs and advancements, the plot underscores the transformative potential of integrating NAS, transfer learning, and compression techniques. This combination enables a paradigm shift, making high-performing models accessible and deployable in resource-constrained blockchain environments, paving the way for robust and scalable anomaly detection solutions.

5. Conclusions

The proposed framework for optimizing Convolutional Neural Networks (CNNs) introduces a groundbreaking approach to addressing the challenges of anomaly detection and transaction analysis in blockchain systems. By integrating Neural Architecture Search (NAS), transfer learning, and model compression techniques, this framework achieves a critical balance between high accuracy and computational efficiency, making it uniquely suited for resource-constrained blockchain environments. NAS automates the design of CNN architectures, ensuring that the resulting models are tailored specifically for the complexities of blockchain datasets, which often involve intricate transaction patterns and anomalies. This automation eliminates the inefficiencies and limitations of manual optimization methods, enabling the exploration of a vast architectural search space to identify the optimal configuration for the task. Transfer learning further enhances the model's adaptability by leveraging pre-trained networks, which significantly accelerates training while improving performance, even in scenarios with limited labeled blockchain data. The incorporation of model compression techniques, particularly filter pruning and quantization, reduces the computational and memory overhead of the CNNs, enabling their deployment in decentralized blockchain nodes with varying resource capacities. These techniques ensure that the framework is not only accurate, but also scalable and deployable in real-world applications.

The efficiency of the proposed approach lies in its holistic integration of these advanced methods. NAS ensures that the architecture is optimized from the ground up, while transfer learning adapts these architectures to domain-specific tasks, and compression techniques refine the final model to meet stringent resource constraints. Together, these components form a pipeline that minimizes computational complexity without compromising the model's analytical capabilities, achieving state-of-the-art results with a test error of 16.5%—the lowest among comparable methods—and a parameter count of just 1.0 M. This combination of accuracy, scalability, and resource efficiency sets the proposed framework apart, making it a robust solution for anomaly detection in blockchain ecosystems. Looking forward, the framework opens several promising directions for future research and development. One perspective involves leveraging federated learning to enable decentralized training of CNNs across blockchain nodes while preserving data privacy. This approach could enhance global model accuracy and robustness, while adhering to the decentralized ethos of blockchain technology. Another perspective focuses on extending the framework to support real-time anomaly detection, addressing the need for low-latency inference in high-throughput blockchain systems, where rapid response times are critical for maintaining network security and performance. Finally, expanding the framework to handle multi-modal blockchain data, such as transaction metadata, smart contract logs, and network activity patterns, would enhance its anomaly detection capabilities, providing a more holistic view of blockchain operations. These perspectives, combined with the current framework's foundation, pave the way for innovative applications of machine learning in blockchain ecosystems, fostering the development of secure, efficient, and intelligent decentralized systems.

Author Contributions: Conceptualization, methodology, and experimentation: H.L.; writing—review and editing: H.L., A.L., E.K. and A.A. All authors have read and agreed to the published version of the manuscript.

Funding: This study is supported via funding from Prince Sattam bin Abdulaziz University project number (PSAU/2025/R/1446).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data can be made available on reasonable request from the corresponding author.

Acknowledgments: This study is supported via funding from Prince Sattam bin Abdulaziz University project number (PSAU/2025/R/1446). The authors would like to acknowledge that this research work was partially financed by Kingdom University, Bahrain, from the research grant number KU-SRU-2024-08.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Workflow of the Proposed Framework for Blockchain Smart Contracts Optimization

The proposed framework, illustrated in Figure 1, provides a systematic approach to optimizing CNNs for anomaly detection and transaction analysis in blockchain smart contracts. The workflow begins with data processing, where raw blockchain data such as transaction logs, contract execution traces, and metadata are preprocessed to extract meaningful features like transaction patterns and anomaly indicators, ensuring a clean and balanced dataset for training. Following this, CNN design leverages Neural Architecture Search (NAS) to automatically generate the most effective architecture for blockchain tasks by iteratively applying crossover and mutation operators; these refine candidate architectures to maximize classification accuracy while minimizing computational complexity. Once the optimal CNN architecture is identified, transfer learning is employed to enhance performance and accelerate training by leveraging pre-trained models, such as ResNet and VGG. This involves freezing general feature-extracting layers while fine-tuning task-specific layers to adapt to blockchain-specific data, significantly reducing training time and improving accuracy. To ensure deployability in resource-constrained blockchain nodes, the next stage, CNN compression, applies techniques like filter pruning to remove redundant filters and quantization to lower precision weights and activations, further reducing computational requirements and memory usage. Additionally, Huffman compression is used to minimize storage needs, making the model even more lightweight. Finally, the framework outputs accurate classifications of blockchain transactions, distinguishing fraudulent or anomalous activities from normal operations, ensuring real-time inference capabilities. This integrated pipeline balances accuracy and efficiency, making it highly adaptable and scalable for diverse blockchain applications.

References

- Bhattacharya, S., & Lane, N. D. (2016, November 14–16). *Sparsification and separation of deep learning layers for constrained resource inference on wearables* [Conference session]. 14th ACM Conference on Embedded Network Sensor Systems CD-ROM (pp. 176–189), Stanford, CA, USA.
- Denton, E., Zaremba, W., Bruna, J., LeCun, Y., & Fergus, R. (2014). Exploiting linear structure within convolutional networks for efficient evaluation. *Advances in Neural Information Processing Systems*, 27, 1269–1277.
- Huynh, D., Gao, Y., Verma, A., & Chan, K. (2017, June 19–23). *DeepMon: Mobile GPU-based deep learning framework for continuous vision applications* [Conference session]. 15th Annual International Conference on Mobile Systems, Applications, and Services (pp. 82–95), Niagara Falls, NY, USA.

- Irvin, J., Rajpurkar, P., Ko, M., Yu, Y., Ciurea-Illcus, S., Chute, C., Marklund, H., Haghighi, B., Ball, R., Shpanskaya, K., & Seekins, J. (2019, January 27–February 1). *Chexpert: A large chest radiograph dataset with uncertainty labels and expert comparison* [Conference session]. Thirty-Third AAAI Conference on Artificial Intelligence, Honolulu, HI, USA, pp. 590–597.
- Islam, M. T., Aowal, M. A., Minhaz, A. T., & Ashraf, K. (2017). Abnormality detection and localization in chest X-rays using deep convolutional neural networks. *arXiv*, arXiv:1705.09850.
- Liang, J., Meyerson, E., Hodjat, B., Fink, D., Mutch, K., & Miikkulainen, R. (2019, July 13–17). *Evolutionary neural automl for deep learning* [Conference session]. Genetic and Evolutionary Computation Conference, Prague, Czech Republic. [\[CrossRef\]](#)
- Liu, Z., Li, J., Shen, Z., Huang, G., Yan, S., & Zhang, C. (2017, October 22–29). *Learning efficient convolutional networks through network slimming* [Conference session]. IEEE International Conference on Computer Vision (pp. 2736–2744), Venice, Italy.
- Louati, H., Bechikh, S., Louati, A., Aldaej, A., & Said, L. B. (2021). Evolutionary Optimization of Convolutional Neural Network Architecture Design for Thoracic X-ray Image Classification. In *International conference on industrial, engineering and other applications of applied intelligent systems* (pp. 121–132). Springer.
- Louati, H., Bechikh, S., Louati, A., Aldaej, A., & Said, L. B. (2022a, July). Evolutionary optimization for CNN compression using thoracic X-ray image classification. In *International conference on industrial, engineering and other applications of applied intelligent systems* (pp. 112–123). Springer International Publishing.
- Louati, H., Louati, A., Bechikh, S., & Ben Said, L. (2022b). Design and Compression Study for Convolutional Neural Networks Based on Evolutionary Optimization for Thoracic X-Ray Image Classification. In *International Conference on Computational Collective Intelligence* (pp. 283–296). Springer International Publishing. [\[CrossRef\]](#)
- Louati, H., Louati, A., Bechikh, S., & Kariri, E. (2024). Joint filter and channel pruning of convolutional neural networks as a bi-level optimization problem. *Memetic Computing*, 16(1), 71–90. [\[CrossRef\]](#)
- Louati, H., Louati, A., Bechikh, S., Masmoudi, F., Aldaej, A., & Kariri, E. (2022c). Topology optimization search of deep convolution neural networks for CT and X-ray image classification. *BMC Medical Imaging*, 22(1), 120. [\[CrossRef\]](#)
- Lu, Z., Whalen, I., Boddeti, V., Dhebar, Y., Deb, K., Goodman, E., & Banzhaf, W. (2019a, July 13–17). *Nsga-net: Neural architecture search using multi-objective genetic algorithm* [Conference session]. Genetic and Evolutionary Computation Conference (pp. 419–427), Prague, Czech Republic.
- Lu, Z., Whalen, I., Dhebar, Y. D., Deb, K., Goodman, E. D., Banzhaf, W., & Boddeti, V. N. (2019b). Multi-criterion evolutionary design of deep convolutional neural networks. *arXiv*, arXiv:1912.01369.
- Lu, Z., Whalen, I., Dhebar, Y. D., Deb, K., Goodman, E. D., Banzhaf, W., & Boddeti, V. N. (2020). Multi-objective evolutionary design of deep convolutional neural networks for image classification. *IEEE Transactions on Evolutionary Computation*, 24(2), 402–415.
- Luo, J. H., Wu, J., & Lin, W. (2017, October 22–29). *ThiNet: A filter level pruning method for deep neural network compression* [Conference session]. IEEE International Conference on Computer Vision (ICCV) (pp. 5058–5066), Venice, Italy.
- Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22, 1345–1359. [\[CrossRef\]](#)
- Sethy, P. K., & Behera, S. K. (2020). Detection of coronavirus disease (COVID-19) based on deep features. *International Journal of Mathematical, Engineering and Management Sciences*, 5(4), 643–651. [\[CrossRef\]](#)
- Shinozaki, T., & Watanabe, S. (2015, April 19–24). *Structure discovery of deep neural network based on evolutionary algorithms* [Conference session]. 2015 IEEE International Conference on Acoustics, Speech and Signal Processing (pp. 4979–4983), Brisbane, Australia.
- Sun, Y., Xue, B., Zhang, M., & Yen, G. G. (2019). Completely automated cnn architecture design based on blocks. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2), 1242–1254. [\[CrossRef\]](#) [\[PubMed\]](#)
- Wang, X., Peng, Y., Lu, L., Lu, Z., Bagheri, M., & Summers, R. M. (2017, July 21–26). *ChestX-ray8: Hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases* [Conference session]. IEEE Conference on Computer Vision and Pattern Recognition (pp. 3462–3471), Honolulu, HI, USA.
- Xie, S., Girshick, R., Dollar, P., Tu, Z., & He, K. (2017, July 21–26). *Aggregated residual transformations for deep neural networks* [Conference session]. IEEE conference on Computer Vision and Pattern Recognition (pp. 1492–1500), Honolulu, HI, USA.
- Yao, L., Poblenz, E., Dagunts, D., Covington, B., Bernard, D., & Lyman, K. (2017). Learning to diagnose from scratch by exploiting dependencies among labels. *arXiv*, arXiv:1710.10501.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.